

Rahnuma

रहनुमा رہنما

A working guide to the ILM, PANINI and Zistgah stack, from bytes and scripts to compilers, Pāṇini and quantum substrates

प्रकृति

	written	Devanagari hub	Romenagri	and back
Devanagari	प्रकृति	→ प्रकृति	→ prak_ri_ti	→ प्रकृति
Telugu	ప్రకృతి	→ प्रकृति	→ prak_ri_ti	→ ప్రకృతి
Bengali	প্রকৃতি	→ प्रकृति	→ prak_ri_ti	→ the same identity
Urdu, as usually written	ہندوی	→ हनदवी	→ _hana_dawee	→ हनदवी
Urdu, with zer and sukun	ہندوی	→ हिन्दवी	→ _hin_dawee	→ हिन्दवी

One identity, many scripts. Telugu and Bengali meet Devanagari in one hub, become the same Romenagri word, and come back exactly. Urdu meets the hub at what is written: without its short-vowel marks the word reads hanadawee; with zer and sukun it is hindawi. Every cell here was produced by the Hindawi tools while this guide was built.

Abhishek Choudhary, AyeAI. Version 2.0.0, 3 October 2026.

Rahnuma, the edition for the field: Generic programmes and qualifications

Contents

Part I: Orientation 2

- 1. How to use this guide 3
- 3. Start here: find your way in 4

Part II: One architecture 6

- 4. Where the work comes from 7

Part IV: Foundations 10

- 17. Numbers inside the machine 11
- 18. Characters inside the machine 13

Part I: Orientation

1. How to use this guide

In plain words. This guide explains a large body of work on computers, languages and intelligence. You can read it, run the programs on your own computer, or ask an AI to help you, and every example in it was really run.

This guide is for anyone who wants to use, check or extend the work gathered in the *zistgah*, *project-ilm* and *hindawiai* organisations on GitHub: students preparing for GATE, engineers who write compilers, firmware or chip layouts, linguists who work with scripts and sound, and researchers in quantum computing and AI. It asks for curiosity and a laptop, and nothing else.

There are four ways in.

- **Understand.** Part II gives the whole architecture first: where the work comes from, how the parts compose, each term with its nearest analogue and the difference, and the three Acts.
- **Read.** Parts IV and V explain the ideas the work rests on, with the mathematics written out: how numbers and letters sit in memory, what makes a transliteration reversible, how Pāṇini's grammar computes, how a compiler turns text into machine instructions, and how an oscillator network and a quantum circuit attack the same problem.
- **Run.** Part III gives the exact commands to set up Linux, Windows with WSL, or macOS, and to run every component of the estate on your own machine.
- **Ask.** Chapter 24 shows how to put any AI assistant to work on these repositories, and how to check what it tells you.

Conventions used throughout:

- Every example that could run while this guide was being built did run, and the text shows its real output, marked *Executed while this guide was built*. The package's own `gate`, `bash ops/verify.sh`, runs those examples again and fails if any output differs from what is printed here.
- Status words are used exactly. **Tested:** a test in the repository asserts it. **Executed:** it ran and its output is shown. **Written:** the code exists but has not been run here. **Mocked:** a stand-in replaces hardware that is not attached. **Planned:** described, not built.
- Everything said about the repositories was retrieved from the repositories themselves on 29 September 2026: their listings on GitHub, their `CITATION.cff` files and their READMEs. Where a repository does not record something, this guide says so instead of filling the gap.
- A dagger, †, marks a statement that follows the author's working notes and write-ups as supplied on 29 September and 3 October 2026, rather than the repositories.
- Sanskrit is transliterated in IAST. The spelling is British.

The work lives at github.com/zistgah, github.com/project-ilm and github.com/hindawiai, on the site zistgah.org, and in the Zenodo records listed in Chapter 12.

3. Start here: find your way in

In plain words. Four steps to find your part of the guide: who you are, your board or field, what you want to do, and your path with a PDF made for you.

This guide serves a school student and a senior researcher; a village health worker and an investor. Nobody needs all of it. Four steps find your part, on the site or with this page:

1. **Who you are.** A school student, a college student, a postgraduate or doctoral researcher; a teacher; an academic researcher; a working professional, someone in public service, policy or law, someone in community, NGO or village work, an artist, writer or maker; an industry partner or investor; or an AI agent working for a person.
2. **Your board or your field.** A school student chooses a board, ICSE, ISC, CBSE, IGCSE, IB, a state board or another; and a class. Everyone else chooses a field from the international standard classification of fields of study, ISCED-F 2013, the same one FAKIR's lattice uses. It places each subject where its own scholars would: political science is under social and behavioural sciences, not the humanities; law is under business, administration and law; linguistics is under languages.
3. **What you want to do.** Learn the basics, run the software, study or research, teach, contribute, or adopt, fund or invest.
4. **Your path and your edition.** The site lists your chapters in order, with reading times and the components to try first, and offers a PDF edition made for you. Every edition below is a printed selection from this one guide, so all of them stay in step.

Edition	PDF
School student	role-school.pdf
College student	role-college.pdf
Postgraduate or doctoral researcher	role-pg.pdf
Teacher or lecturer	role-teacher.pdf
Academic researcher	role-researcher.pdf
Working professional or engineer	role-professional.pdf
Public service, policy or law	role-public.pdf
Community, NGO or village work	role-community.pdf
Artist, writer or maker	role-creator.pdf
Industry partner, investor or funder	role-industry.pdf
Generic programmes and qualifications	field-00.pdf
Education	field-01.pdf
Arts and humanities	field-02.pdf
Social sciences, journalism and information	field-03.pdf

Edition	PDF
Business, administration and law	field-04.pdf
Natural sciences, mathematics and statistics	field-05.pdf
Information and Communication Technologies (ICTs)	field-06.pdf
Engineering, manufacturing and construction	field-07.pdf
Agriculture, forestry, fisheries and veterinary	field-08.pdf
Health and welfare	field-09.pdf
Services	field-10.pdf

The first public version of this guide, written for advanced researchers and senior professionals, stays as it was at [version 1](#).

Part II: One architecture

4. Where the work comes from

In plain words. This work began long before today's AI. It grew along five connected lines: languages and computers, learning machines, robots and medicine, organised knowledge, and the future of people and their homes.

This work did not begin with today's language models, and it is not a collection of separate projects. It is one architecture built along several lines of work that started at different times, and it reads correctly only in that order. The dated record, with the place where each entry can be checked, is the next chapter; this one gives the lines themselves.

Five lines that intersect

Linguistic and computational identity. Romenagri (2003), the reversible ASCII-7 kernel, came first; the Hindawi Programming System (15 August 2004) built a complete systems-programming stack on it; ILM, Integrative Linguistic Multiscript, succeeds Hindawi and generalises it to every script and language; PANINI's construct model now sits above ILM as the common core of programming, prompting and realization.

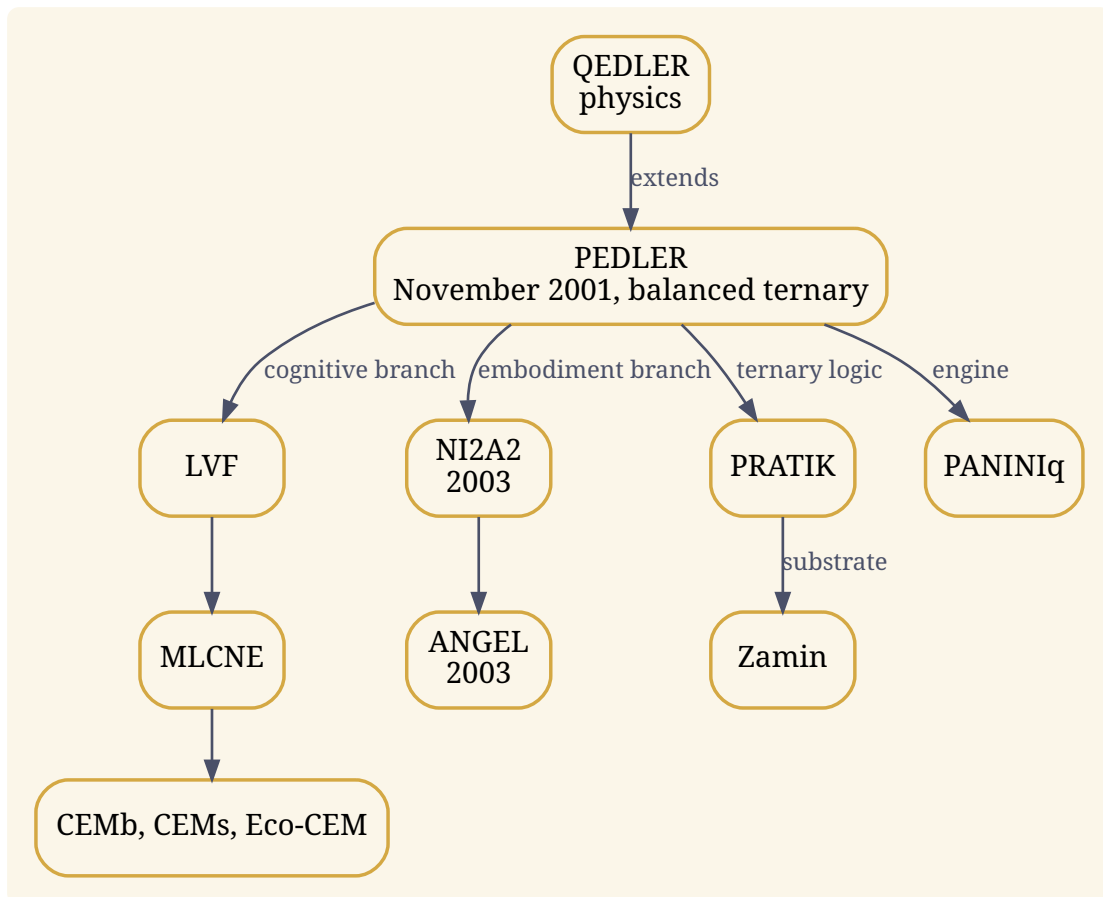


Figure 1. PEDLER's branches: cognition, embodiment, physics and silicon.

Cognition and event-driven computation. PEDLER, the Point Event-Driven Learner (November 2001, Indian patent application 3033/CHE/2011), is not one genealogy but two branches: a cognitive branch through LVF and MLCNE to the cognitive enablement modules, CEMb and CEMs, and an embodiment

branch through NI2A2 (2003) to the cognitive robot ANGEL (2003). QEDLER carries PEDLER into physics. PRATIK and Zamin take its balanced-ternary, event-driven logic towards silicon.

Embodiment, medicine and field systems. HMSEI, a medical wearable (2002); ANGEL; TARA; RDK, among Nokia's global top ten in 2010; Dr Rho, the TARA and RDK medical telepresence platform, Highly Commended at the IET Innovation Awards in 2014; GramSheel's Village Knowledge Center; a Stockholm Challenge finalist in 2010. The AyeAI Triad closes this line formally: AyeAM for embodiment, AyeAI for cognition, AyeCNSe for coordination and communication.

Knowledge and reasoning. UKOP, the reference architecture for human knowledge across ISIC, ISCO and ISCED; FAKIR, its kernel; Dhancha, the spine every domain is built on; the domains themselves; and the working layer that runs on them: GENIE, the cyclers, AAB and Mez.

Continuity and civilisation. Humanesque, its constitutional derivation from the Proclamation of Individual Equity, TransEg, the transferred alter ego, and the habitats on Earth, the Moon and Mars.

The lines cross constantly. PEDLER supplies the Act, the executable intent that the constitutional corpus is built from; Romenagri and ILM carry every language into FAKIR's lattice; PANINI runs the cyclers and also drives the realization compilers; TransEg receives what yadein records. That is why the chapters that follow teach the pieces as nodes of one graph rather than as a list.

The invariant

† What the architecture preserves, whatever changes around it, is meaning, intent, identity, agency, capability, provenance, equity and continuity, while the representation, the language, the script, the formalism, the machine, the substrate, the embodiment, the social setting and eventually the habitat all change. Romenagri is the smallest instance of the principle: a name keeps its identity through every tool of a toolchain. TransEg is the largest: a person's constituted identity keeps its continuity beyond the substrate that first carried it.

Sovereignty and equity

The Proclamation of Individual Equity, PoIE ([zistgah/poie](https://doi.org/10.5281/zenodo.21397274), doi:10.5281/zenodo.21397274), makes the individual the primary locus of equity, identity and agency. Sovereignty is recursive from there: individual, family, community, institution, province, nation and beyond, across political, economic, social and technological dimensions. Technological sovereignty, in this sense, is the capacity to determine, govern, develop, reproduce, modify and deploy one's own domain; the precise, testable form of it for software is given in Chapter 21. The aim that follows is enablement: that people become producers, builders, researchers and owners of computing in their own languages, not permanent consumers of systems built elsewhere.

The orientation map

His poster "The Expanse of Our Epistemic Realms" runs one axis from the inner to the outer: being, meaning, knowledge, language, cognition, code, systems, society, civilisation, planet. At its apex stands Kaivalyik AGI; beneath it, the Equitable Mind, one humanity with one identity and infinite

expressions; and the rule that every sentient individual, bioform or synthetiform, is a sovereign. Read the rest of this Part along that axis: language and identity first, then computation, cognition, knowledge and agency, realization, and the Acts that carry them to civilisational scale.

Part IV: Foundations

17. Numbers inside the machine

In plain words. How a computer stores numbers using only ones and zeros, including negative numbers and fractions.

A processor holds voltages that stand for bits. Everything else on this page, including the hexadecimal digits, is notation for people.

Positional notation

A number written with digits d_i in base b is

$$n = \sum_{i=0}^{k-1} d_i b^i, \quad 0 \leq d_i < b.$$

So $42 = 101010_2 = 2A_{16}$: the letters A to F are simply names for the values ten to fifteen, chosen so that one hexadecimal digit covers exactly four bits. The machine never sees the letter A, any more than it sees the letters of a keyword in a program.

Signed integers: two's complement

With n bits $b_{n-1} \dots b_0$, the value is

$$v = -b_{n-1} 2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i, \quad -2^{n-1} \leq v \leq 2^{n-1} - 1.$$

Negative x is therefore stored as $2^n - |x|$. The payoff is that one adder serves signed and unsigned arithmetic alike, which is why every processor in use works this way.

Reals: IEEE 754

A double-precision number has a sign bit s , eleven exponent bits e and fifty-two fraction bits f , and for normal numbers

$$v = (-1)^s \times (1.f)_2 \times 2^{e-1023}.$$

One tenth has no finite binary expansion, so it is stored rounded, and sums of rounded values are rounded again.

Byte order and bit order

A 32-bit value occupies four bytes. A little-endian machine (x86, and ARM as usually configured) stores the least significant byte at the lowest address; a big-endian machine stores the most significant byte first, and network protocols use big-endian, which is why C has `htonl` and `ntohl`. That is endianness, and it concerns bytes.

Bit numbering is a separate convention: whether bit 0 is the least or the most significant bit. The hexadecimal digit E is **1110** when written most significant bit first and **0111** when written least significant bit first, and a serial line such as a UART sends the least significant bit first. Both conventions are representations. The circuit has only the wires.

Executed while this guide was built: `python3 examples/numbers_demo.py`

Integers and reals

```
0x0A0B0C0D little-endian bytes: 0d 0c 0b 0a    big-endian: 0a 0b 0c 0d
-5 in 8-bit two's complement: 11111011    (256 - 5 = 251)
0.1 as an IEEE 754 double: 3fb999999999999a = 0x1.999999999999ap-4
0.1 + 0.2 == 0.3 is False; the sum is 0.30000000000000004
```

These are the facts the GATE syllabus asks for under number representation and computer arithmetic, fixed and floating point (Chapter 26).

18. Characters inside the machine

In plain words. How letters from any script become numbers inside a computer, and why a name must survive every tool a program passes through.

ASCII

Seven bits, 128 codes: 0 to 31 and 127 are control codes, 32 to 126 are printable, and **A** is 65, or 0x41. It was designed in the 1960s for English teleprinters, and nearly every later character code keeps it as its first 128 positions.

ISCII

ISCII, the Indian Script Code for Information Interchange (IS 13194:1991), is an 8-bit code. Codes 0 to 127 are ASCII. The upper half holds one layout shared by the Brahmi-derived scripts of India: the same code means KA in Devanagari, Bengali, Gurmukhi, Gujarati, Oriya, Tamil, Telugu, Kannada and Malayalam, and an attribute code selects the script for display. The shared layout records the common phonological organisation of these scripts. The price is that the script, and with it the language, has to be known from context.

Unicode

Unicode has room for 1,114,112 code points, U+0000 to U+10FFFF, in 17 planes of 65,536. Plane 0, the Basic Multilingual Plane, holds every major Indic block. Each script has its own block, so KA has a different code point in each. The blocks were laid out from ISCII's order, however, so KA sits at offset 0x15 and the virama at offset 0x4D in every one of the nine: the commonality ISCII carried in a single code, Unicode carries as a fixed offset.

UTF-8

A code point U is written in one to four bytes:

$$\begin{aligned} U < 2^7 : & \quad 0xxxxxxx \\ U < 2^{11} : & \quad 110xxxxx \ 10xxxxxx \\ U < 2^{16} : & \quad 1110xxxx \ 10xxxxxx \ 10xxxxxx \\ U < 2^{21} : & \quad 11110xxx \ 10xxxxxx \ 10xxxxxx \ 10xxxxxx \end{aligned}$$

The bits of U fill the **x** positions from the left. Every Devanagari code point lies between 2^{11} and 2^{16} , so it takes three bytes in UTF-8 against two in UTF-16. Because no continuation byte (**10xxxxxx**) can be mistaken for a leading byte, a reader can find character boundaries from any point in a stream, and an ASCII byte never appears inside a multi-byte character, which is why UTF-8 passes safely through tools written for ASCII.

Executed while this guide was built: `python3 examples/chars_demo.py`

1. The same letter KA in nine Unicode blocks: every block keeps ISCII's order, offset 0x15

Devanagari	block U+0900	KA U+0915 क	virama U+094D
Bengali	block U+0980	KA U+0995 ঙ	virama U+09CD
Gurmukhi	block U+0A00	KA U+0A15 ਙ	virama U+0A4D
Gujarati	block U+0A80	KA U+0A95 ૙	virama U+0ACD
Oriya	block U+0B00	KA U+0B15 ଙ	virama U+0B4D
Tamil	block U+0B80	KA U+0B95 ங	virama U+0BCD
Telugu	block U+0C00	KA U+0C15 ఙ	virama U+0C4D
Kannada	block U+0C80	KA U+0C95 ೙	virama U+0CCD
Malayalam	block U+0D00	KA U+0D15 ങ	virama U+0D4D

2. UTF-8 by hand for U+0915 (KA): 16 bits split 4 + 6 + 6 into 1110xxxx 10xxxxxx 10xxxxxx
0000 100100 010101 -> 11100000 10100100 10010101 = E0 A4 95
Python agrees: E0 A4 95
the Devanagari block U+0900 to U+097F is E0 A4 80 to E0 A5 BF

3. Words are sequences of code points; aksharas are clusters of them
प्रकृति: 7 code points, 21 UTF-8 bytes: U+092A U+094D U+0930 U+0915 U+0943 U+0924 U+093F
पितृ: 4 code points, 12 UTF-8 bytes: U+092A U+093F U+0924 U+0943
क्षि: 4 code points, 12 UTF-8 bytes: U+0915 U+094D U+0937 U+093F
categories in क्षि: U+0915 Lo, U+094D Mn, U+0937 Lo, U+093F Mc

4. Normalisation: QA U+0958 decomposes, and composition does not rebuild it
NFD(U+0958) = U+0915 U+093C
NFC(that) = U+0915 U+093C
equal as strings before normalising: False; after NFC on both: True

What the output shows

Clusters. A reader sees क्षि as one unit, a grapheme cluster in the terms of Unicode's UAX #29, but it is four code points: KA (a letter, category Lo), VIRAMA (a nonspacing mark, Mn), SSA (Lo) and VOWEL SIGN I (a spacing mark, Mc). The vowel sign is stored after the consonant and drawn before it. Reordering and conjunct formation happen when text is rendered, in a shaping engine such as HarfBuzz working from the font's OpenType tables. Storage order is phonetic; display order is visual. Cursor movement, deletion, search and string length therefore have to work on clusters, not on code points.

Normalisation. QA has its own code point, U+0958, and also a decomposed spelling, KA followed by NUKTA (U+0915 U+093C). The two look identical and compare unequal until normalised, and because U+0958 is a composition exclusion, normalising to NFC yields the decomposed form. A lexer that accepts identifiers in Indic scripts has to normalise them, or two names that look the same will be different names.

Joiners. ZERO WIDTH JOINER and ZERO WIDTH NON-JOINER (U+200D, U+200C) ask for, or prevent, a joined form: KA, VIRAMA, ZWJ, SSA requests a half-form of KA, and KA, VIRAMA, ZWNJ, SSA requests a visible virama. They are invisible, and invisible characters are a classic route for impersonation.

Confusables. Characters from different scripts that look alike, such as Latin **a** and Cyrillic **а**, let one name pose as another. Unicode's UTS #39 defines how to detect confusable strings, with its data in `confusables.txt`, and sets rules for mixing scripts in one identifier. Any system that accepts identifiers in several scripts should apply them.

A name has to survive the whole toolchain

Python 3 accepts identifiers in any script under PEP 3131, and GCC has accepted UTF-8 identifiers in C since version 10. That settles only the first step, and for systems work it settles the wrong one.

A keyword disappears during compilation. By the time a program is machine code, a loop written with `for` and the same loop written with `क्रम` have both become a comparison and a branch (Chapter 21). A name does not disappear. Every function, global variable, parameter and register mapping lives on: in the object file's symbol table, `.symtab` with its strings in `.strtab`; in the DWARF debugging information; in linker scripts and map files; in the kernel's own symbol table, from which a panic trace is printed; and in every tool a developer points at the binary: `nm`, `objdump`, `addr2line`, GDB, OpenOCD driving a JTAG or SWD probe on a microcontroller; a vendor's IDE. Each of these has its own rules about which bytes a symbol may contain, and the one alphabet every one of them accepts is letters, digits and the underscore.

So the question for firmware, kernels and microcontrollers is not whether a compiler accepts a name written in Devanagari. It is whether that name comes back, byte for byte, at every one of those tools, and can still be read against the source. Hindawi's answer, since 2003, is Romenagri: every name travels as a word over A to Z, a to z and the underscore, which every tool in the chain accepts, and the mapping is bijective, so the name in the script can be recovered from the binary at any point. Chapter 19 sets out the scheme and Chapter 21 runs it through GCC, the ELF symbol table, DWARF and GDB.

Composing conjuncts

Hindawi's APCISR composed half-consonant conjuncts of any length in software, in the years when display hardware for Indian scripts had fixed limits on them. Shaping engines solve the same problem today. It still matters wherever text must be shown without a full shaping stack, for example on a boot console before an operating system has loaded its fonts.

© 1993–2026 Abhishek Choudhary. All rights reserved. AyeAI. Text CC BY-SA 4.0; scripts GPL-3.0-or-later. Version 2.0.0, 3 October 2026.